

Universidad de Navarra

# Big Data & Cloud Computing

## Entrega 1

**Aimar Luis, Telmo López y Fátima Michel**

**Link Repo: https://github.com/AimarLuis/Big-Data-MITMA**

*Arquitectura Big Data local sobre la matriz de movilidad del MITMA (26-10-2022 a 01-11-2022)*

# 1. Planteamiento del problema y arquitectura

En este proyecto se utilizó un data set del MITMA para el estudio de movilidad en España completo durante 1 semana del 26 de Octubre del 2022 al 1 de Noviembre del 2022.

El dataset contiene las siguientes columnas: fecha, periodo (franja horaria), origen, destino, distancia, actividad de origen, actividad de destino, estudio en origen posible, estudio en destino posible, residencia, renta, edad, sexo, número de viajes (viajes) y kilómetros totales (viajes_km). El esquema se declara explícitamente con StructType, como se puede ver en `VIAJES_SCHEMA` (Figura 1):

```
VIAJES_SCHEMA = StructType([
    StructField("fecha", StringType(), True),              # 20221026 -> se convierte a fecha
    StructField("periodo", IntegerType(), True),           # hora de inicio: "00".."23" -> 0..23
    StructField("origen", StringType(), True),             # código INE de municipio (o FR..., PT..., externo)
    StructField("destino", StringType(), True),
    StructField("distancia", StringType(), True),          # banda: 0.5-2, 2-10, 10-50, >50 km
    StructField("actividad_origen", StringType(), True),   # casa, trabajo_estudio, frecuente, no_frecuente
    StructField("actividad_destino", StringType(), True),
    StructField("estudio_origen_posible", StringType(), True),  # si/no
    StructField("estudio_destino_posible", StringType(), True),
    StructField("residencia", StringType(), True),         # provincia de residencia (2 dígitos)
    StructField("renta", StringType(), True),              # <10, 10-15, >15 (miles de euros)
    StructField("edad", StringType(), True),               # 0-25, 25-45, 45-65, 65-100, NA
    StructField("sexo", StringType(), True),               # hombre, mujer, NA
    StructField("viajes", DoubleType(), True),             # estimación de MITMA (decimal)
    StructField("viajes_km", DoubleType(), True),          # viajes x km: NUNCA como predictor (fuga)
])
```

**Figura 1.** *Esquema explícito de la tabla de viajes (`src/etl.py`). Nunca se usa `inferSchema`.*

Además de la tabla principal se usan tres tablas auxiliares:

- **Zonificación de municipios de MITMA:** relaciona el código de municipio con su nombre. La clave del join es el código de municipio.
- **Provincias (INE):** relaciona el código de provincia con su nombre y su comunidad autónoma. El código de provincia son los dos primeros dígitos del código INE del municipio, así que no hace falta modificar la tabla de MITMA.
- **Calendario:** construida por el equipo, indica el tipo de cada día (laborable, fin de semana o festivo). El 1 de noviembre es festivo nacional.

Para el caso de Machine Learning se busca predecir los viajes medios diarios que salen de cada municipio de origen para cada rango de edad. Para ello se emplea una regresión lineal de MLlib, cuyo rendimiento se compara con dos líneas base sencillas: la media global y la media de cada municipio.

Las tres preguntas que este análisis va a responder son las siguientes:

1. **(Agrupación)** ¿Cuántos viajes medios diarios genera cada provincia de origen según el tipo de día (laborable, fin de semana o festivo)?
2. **(Ventana — ranking)** ¿Cuáles son los 3 municipios que más viajes reciben cada día?
3. **(Ventana — comparación contra el grupo)** Para cada municipio y día, ¿cuánto se desvía su número de viajes respecto a la media semanal de ese municipio, y cuánto varía respecto al día anterior? (detecta picos o caídas anómalas de movilidad)

Respecto a la inscripción, el periodo se reduce de dos meses a una semana (unos 7 GB, manejables en el entorno local), y la pregunta por día de la semana se cambia por la de tipo de día, porque con una sola semana cada día de la semana aparece una única vez.

## 1.1 Análisis de las 5V

**Volumen:** Si aplica en este caso ya que el dataset que utilizamos pesa al final 6,87 GB, con 78.340.906 filas en 7 días (unos 11,2 millones de filas al día). Cada fila es un flujo origen→destino por franja horaria y segmento (actividad, renta, edad, sexo), y esa combinación de dimensiones multiplica el número de registros. No se puede poner en memoria con herramientas como pandas en un equipo de 8 GB de RAM, y los joins y las funciones de ventana sobre el conjunto completo por lo tanto se utiliza un data distributed system (Spark) y un formato columnar particionado (Parquet por fecha).

**Velocity:** En este caso no aplica ya que estos datos no se utilizan en el tiempo real, en vez MITMA los publica como ficheros diarios pero no en tiempo real, si no con retrasos ya que los procesa y luego nuestro análisis es sobre una semana de 2022. No hay flujo en tiempo real ni requisito de latencia por debajo del día. Por eso el procesamiento es batch. Se procesan en batch para poder sacar análisis significativo para la mejora de movilidad.

**Variety:** No aplica en este caso. Todos los datos son estructurados y tabulares, en un único formato (CSV separado por |, 15 columnas fijas), y las tablas auxiliares también son CSV. No hay texto libre, imágenes ni JSON. Lo más heterogéneo son los códigos de zona, que mezclan códigos INE (01001), agregaciones de municipios de MITMA (01004_AM) y zonas extranjeras (FR101, PT…, externo), que son el 0,90 % de las filas. Esto se resuelve con una regla en la ETL.

**Veracity:** Si aplica ya que la veracidad es la calidad y exactitud de los datos, en este caso vienen de MITMA. Los viajes no son un conteo directo sino una estimación decimal que MITMA calcula a partir de datos de telefonía móvil y extrapola a la población. Además, la edad está suprimida (NA) en el 30,2 % de las filas y el sexo en el 51,8 %, por privacidad.

**Value:** El value si aplica a este caso ya que este análisis nos sirve para entender patrones de movilidad tales como municipios que atraen más viajes, diferencias entre días laborables y fines de semana, desplazamientos por provincia, etc. Esto en el futuro puede ayudar a mejorar la movilidad en España. Para el modelado, permite construir un modelo de regresión lineal que predice viajes según el municipio de origen y el rango de edad. Estos resultados son útiles para la planificación de transporte e infraestructuras, turismo y gestión de eventos.

## 1.2 Architecture

El caso hipotético que utilizamos fue en el cual **MovilidadData**, una consultora que asesora a ayuntamientos y operadores de transporte, y que necesita un dashboard de movilidad actualizado cada día más consultas ad-hoc sobre todo el histórico. En este caso tenemos:

- **Fuente:** un fichero CSV diario de viajes entre municipios (como los del MITMA), que llega una vez al día con retraso. En nuestro proyecto, 7 días ≈ 6,87 GB.
- **Consumo 1, dashboard:** viajes por destino y franja de edad del día anterior, listo cada mañana. No se necesita tiempo real: un retraso de horas es aceptable. Se procesa en batch.
- **Consumo 2, ML y data engineers:** analistas que hacen preguntas nuevas sobre meses o años de histórico (picos por festivo, municipios con más crecimiento) y que además quieren crear modelos (regresión sobre municipio y edad).
- **Restricciones:** el esquema de los ficheros puede cambiar, el histórico solo crece (nunca se borra) y el presupuesto es limitado, así que se prefiere hardware barato y almacenamiento abierto (Parquet).

Elegimos un Data Lake con una capa curada y ordenada por encima, porque el caso mezcla un dashboard fijo con preguntas que aún no conocemos y con machine learning, y una arquitectura Lambda con solo la capa de batch, porque nos interesa más la precisión que la velocidad.

**Decisión 1: Data Lake frente a Data Warehouse**

Se elige una arquitectura de **Data Lake** sobre HDFS, organizada en tres zonas: raw (los CSV originales sin tocar), curada (Parquet limpio y particionado por fecha) y resultados. El caso de uso lo justifica por cuatro razones:

- **Esquema que cambia.** Como el esquema de los ficheros puede cambiar, conviene guardar los datos originales tal cual llegan y aplicar el esquema al leerlos (*schema-on-read*). En un Data Warehouse el esquema se fija antes de cargar (*schema-on-write*), y cada cambio en la fuente obliga a rediseñar las tablas y los procesos de carga.
- **Reprocesar el histórico.** Al conservar la zona raw intacta, si cambia la lógica de limpieza o surge una pregunta nueva, se puede recalcular todo desde el origen.
- **Machine Learning.** Los data engineers necesitan acceder a los datos detallados con Spark y MLlib, no sólo a tablas agregadas para informes, que es para lo que está pensado un Data Warehouse.
- **Coste.** HDFS funciona sobre hardware barato y Parquet es un formato abierto, que no ata a ningún proveedor.

**Inconvenientes:** un Data Lake no garantiza por sí solo la calidad ni la coherencia de los datos. Sin control puede convertirse en un "data swamp" con datos duplicados, mal tipados o sin documentar. Además, las consultas del dashboard serían más inmediatas en un Data Warehouse, con índices y SQL optimizado. Para reducir esta inconveniencia, el pipeline separa en zonas, aplica un esquema explícito (Struct Type) y ejecuta controles de calidad con acción definida en cada ejecución. El dashboard se alimenta de resultados ya agregados en la zona de resultados. Esta combinación se acerca a la idea de un *lakehouse*: almacenamiento barato y abierto con una capa curada fiable para consultar.

**Decisión 2: Lambda frente a Kappa**

Kappa trata todos los datos como un flujo continuo (*streaming*); Lambda combina una capa *batch*, que procesa todo el histórico, con una capa de velocidad para el tiempo real. En este caso de uso los datos llegan **una vez al día como un fichero** y el dashboard admite horas de retraso, así que no hay ninguna necesidad de tiempo real. Por eso se adopta un enfoque **Lambda reducido a su capa batch**: cada día se procesa el nuevo fichero junto con el histórico, sin capa de velocidad.

**Ventajas:**

- **Encaja con los datos:** los datos son ficheros diarios, no eventos continuos.
- **Simplicidad:** Kappa exigiría montar una infraestructura de streaming (por ejemplo Kafka) para unos datos que no llegan en streaming, con más complejidad y coste sin beneficio.
- **Calidad de análisis:** al ser batch, los datos se procesan con más tiempo y con mayor precisión.

**Inconvenientes:**

- **Sin tiempo real:** si el cliente pidiera datos en tiempo real, habría que añadir una capa de velocidad. En Lambda eso implica mantener dos caminos de código (batch y velocidad) que deben dar resultados coherentes, que es la principal crítica a esta arquitectura.
- **Retraso:** los resultados siempre van con al menos un día de retraso.

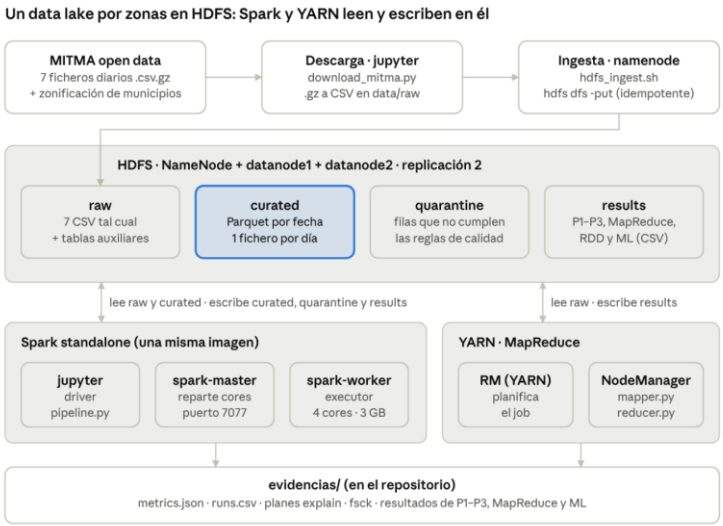



**Figura 2.** *Arquitectura implementada: fuente, ingesta, zonas de HDFS y los dos motores de cálculo (`docs/arquitectura.png`).*

El diagrama muestra exactamente los servicios del docker-compose.yml y las zonas que crea el pipeline. Los datos entran una vez por la ingesta y se quedan en HDFS; Spark y YARN leen de las zonas y escriben en ellas, y las cifras acaban en evidencias/. El punto de entrada es el proceso que se ejecutaría cada día para incorporar el nuevo fichero.

## 1.3 Entorno Docker

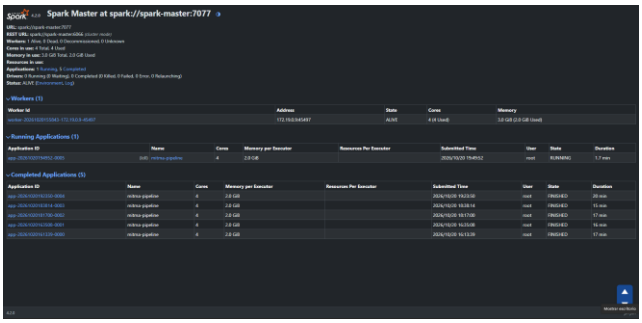
Todos los servicios se ejecutan como contenedores de Docker en una misma red, levantados con un solo comando, `docker compose up -d --build`. Desde un clon limpio no hace falta ningún paso manual: los contenedores se encuentran por su nombre (namenode, spark-master, jupyter).

| Servicio | Imagen | Puertos | Función |
|---|---|---|---|
| namenode | bde2020/hadoop-namenode:2.0.0-hadoop3.2.1-java8 | 9870, 9000 | Índice de HDFS. Monta el repositorio para la ingesta, fsck y MapReduce |
| datanode1, datanode2 | bde2020/hadoop-datanode | — | Guardan los bloques. Replicación 2 |
| resourcemanager | bde2020/hadoop-resourcemanager | 8088 | YARN: planifica el job de MapReduce |
| nodemanager | Imagen del curso + python3 | — | Ejecuta las tareas de MapReduce (mapper.py, reducer.py) |
| historyserver | bde2020/hadoop-historyserver | — | Historial de jobs de YARN |
| spark-master | quay.io/jupyter/pyspark-notebook | 8080 | Master del clúster Spark standalone |
| spark-worker | quay.io/jupyter/pyspark-notebook | 8081 | Worker: 4 cores, 3 GB |
| jupyter | quay.io/jupyter/pyspark-notebook | 8888, 4040 | Entorno PySpark y driver del pipeline |

Docker tiene 8 GB asignados. El master, el worker y Jupyter usan la misma imagen, para garantizar las mismas versiones de Spark y Python en el driver y en los executors. El NodeManager se amplía con Python 3, necesario para ejecutar el mapper y el reducer de Hadoop Streaming. Spark y MapReduce nunca se ejecutan a la vez: el punto de entrada los lanza en secuencia.

El entorno de desarrollo está conectado al clúster: el *smoke test* del pipeline escribe y vuelve a leer 5 filas en HDFS desde el driver (Jupyter) a través del worker (`metrics.json → a3_smoke`). En la Figura 3 se ven los dos DataNodes en servicio y en la Figura 4 el worker de 4 cores registrado en el master con las aplicaciones `mitma-pipeline`.




**Figuras 3 y 4.** *Izquierda (Figura 3): NameNode (`localhost:9870`) con datanode1 y datanode2 en servicio. Derecha (Figura 4): Spark master (`localhost:8080`) con 1 worker de 4 cores y 3 GB y las aplicaciones del pipeline (`evidencias/capturas/a3_*`).*

## 1.4 Metodología de trabajo

La metodología de trabajo fue que creamos dos diferentes versiones pero con la misma idea de architectura, mismas preguntas y baseline. Luego se comparó las ventajas y desventajas de cada uno verificando que todos funcionaran. Al final nos quedamos con una versión y esa versión se modificó agregando también algunos módulos de la otra versión.

# 2. Almacenamiento en HDFS

## 2.1 Zonas y Parquet particionado

Los datos se organizan en HDFS siguiendo una estructura de *data lake* con 5 zonas, cuyas rutas se definen en `config/pipeline.env`:

| Zona | Ruta | Contenido | Cómo se escribe |
|---|---|---|---|
| Raw | /raw/mitma/viajes | Los 7 CSV diarios, sin modificar | hdfs dfs -put desde scripts/hdfs_ingest.sh; nunca se sobrescribe un fichero completo |
| Raw (auxiliares) | /raw/mitma/municipios, /raw/mitma/aux | Zonificación de MITMA, provincias y calendario | Igual |
| Curated | /curated/mitma/viajes/fecha=AAAA-MM-DD | Parquet limpio, tipado y con columnas derivadas | repartition(fecha) + partitionBy(fecha) + sobrescritura dinámica |
| Quarantine | /quarantine/mitma/viajes | Filas que no cumplen las reglas de calidad (0 filas en esta semana) | overwrite |
| Results | /results/mitma/... | Resultados de P1–P3, MapReduce, RDD y predicciones de ML | overwrite, coalesce(1) en CSV |

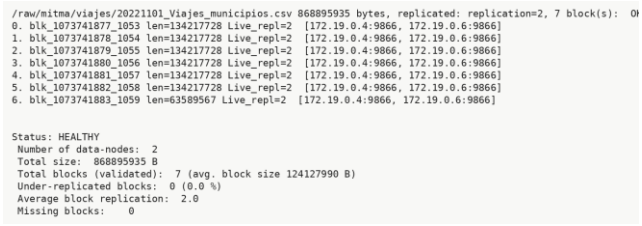
Al ingresar, se copian los ficheros desde el folder data/raw a la zona raw. Es idempotente, es decir antes de subir cada fichero comprueba si ya existe en HDFS con el mismo tamaño y en ese caso, lo salta. Por lo tanto, no hay datos duplicados. La zona raw contiene 6,87 GB (6,4 GiB) en 10 ficheros: 7 CSV diarios y las 3 tablas auxiliares (`evidencias/hdfs_ls_R.txt`).

La **zona curated** se escribe en Parquet particionando por la columna fecha. Por lo tanto, se crea una carpeta por día. Antes de escribir se aplica repartition("fecha"), de forma que todas las filas de un mismo día las procesa una sola tarea y cada día queda en **un único fichero** de entre 124 y 153 MB. Sin este paso, cada día se repartía en 32 ficheros de unos 5 MB (uno por cada partición de shuffle, ver apartado 6), un ejemplo del problema de los ficheros pequeños: muchos ficheros por debajo del tamaño de bloque sobrecargan al NameNode, que guarda en memoria los metadatos de cada fichero y cada bloque. Particionar por fecha permite además que las consultas que filtran por día lean solo las carpetas necesarias (*partition pruning*).

## 2.2 Bloques y replicación

La zona curated contiene 7 ficheros Parquet (uno por carpeta de fecha) que suman 991.882.954 bytes (0,99 GB, que HDFS muestra como 945,9 MiB) repartidos en 13 bloques. Cada fichero mide entre 124 y 153 MB: los seis que superan los 128 MB ocupan 2 bloques (uno lleno y otro con el resto) y el del 1 de noviembre, el festivo y el día con menos viajes, cabe en uno solo. Por eso el tamaño medio de bloque baja a 76 MB. Los 13 bloques tienen sus 2 réplicas (Average block replication: 2.0), no hay ninguno under-replicated y el estado es HEALTHY. Esto confirma también que el repartition("fecha") evita los ficheros pequeños: un fichero por día, de tamaño cercano al bloque de HDFS.

Al hacer un: `hdfs fsck /curated/mitma/viajes -files -blocks -locations` obtenemos desde `evidencias/fsck_curated.txt` la Figura 6. En la Figura 5 está el mismo comando sobre un CSV de raw (`evidencias/fsck_raw_un_dia.txt`): el fichero del 1 de noviembre (868.895.935 bytes) se divide en 7 bloques de 128 MB (el último de 60,6 MB), cada uno con sus dos réplicas, una en cada DataNode (`Live_repl=2`).







**Figuras 5 y 6.** *Salida de `hdfs fsck -files -blocks -locations` sobre el CSV de raw del 1 de noviembre (izquierda) y sobre toda la zona curated (derecha).*

## 2.3 Parquet

La replicación se ve también en `evidencias/hdfs_du.txt`: cada zona ocupa en disco el doble que sus datos. HDFS muestra los tamaños en unidades binarias (1 G = 1.073.741.824 bytes), por eso los 6,87 GB de raw aparecen como 6.4 G y los 0,99 GB de curated como 945.9 M.

```
6.4 G    12.8 G  /raw/mitma/viajes
945.9 M  1.8 G   /curated/mitma/viajes
1.8 K    3.6 K   /quarantine/mitma/viajes
```

**Comparación de tiempos:** se lanza la misma consulta sobre el CSV de raw y sobre el Parquet de curated, 3 veces cada una y sin caché, y se toma la mediana. Se ejecuta con `write.format("noop")`, que ejecuta la consulta entera sin traer datos al driver. La consulta A suma los viajes por día (todas las filas, 2 columnas) y la B lo mismo filtrando un solo día (`src/formats.py`, `metrics.json → b2_formatos`).

| | CSV (raw) | Parquet (curated) | Diferencia |
|---|---|---|---|
| Tamaño en HDFS | 6,87 GB (6,4 GiB) | 0,99 GB (945,9 MiB) | 6,9 veces menos |
| A · todas las filas, 2 columnas | 43,57 s | 1,01 s | 43 veces más rápido |
| B · un solo día | 43,72 s | 1,53 s | 29 veces más rápido |
| Particiones al leer los 7 días | 53 | 8 | |

Parquet ocupa unas 7 veces menos y la consulta es entre 29 y 43 veces más rápida. Además, el Parquet curado tiene más columnas que el CSV original (las columnas derivadas: las marcas de zona extranjera y los códigos de provincia), y aun así ocupa mucho menos.

Spark se comporta de esta manera ya que:

- **El shema viaja con los datos:** Cada fichero Parquet guarda su esquema y los tipos de cada columna. Al leerlo, Spark no tiene que convertir texto a números o fechas, como ocurre con el CSV.
- **Formato columnar y Lectura selectiva de columnas:** Parquet guarda los datos por columnas, así que Spark lee del disco solo las columnas que necesita la consulta, en este caso fecha y viajes, e ignora el resto.
- **Compresión.** Al almacenar juntos los valores de una misma columna, que son del mismo tipo y muy repetitivos, se comprimen mucho mejor. Se usa compresión Snappy. Esto explica la reducción de 6,87 GB a 0,99 GB.
- **Particionado por fecha.** Al estar organizado en una carpeta por día, una consulta que filtre por fecha solo lee las carpetas necesarias (*pruning*). En el plan de la consulta B aparece `PartitionFilters: [isnotnull(fecha), (fecha = 2022-10-26)]` (`evidencias/plan_b2_parquet_un_dia.txt`).

Por último, gzip no es divisible: un día en `.csv.gz` se lee con 1 sola partición (5,50 s para contarlo), frente a las 8 particiones del mismo día descomprimido (1,04 s). Por eso los ficheros se descomprimen antes de subirlos a raw (`metrics.json → b2_gzip_vs_csv_un_dia`).

# 3. Map Reduce

El job calcula el **total de viajes por provincia de origen y día** en los 7 días, con la clave `PP|AAAAMMDD`. Se implementa con Hadoop Streaming, que permite escribir el mapper y el reducer en Python: Hadoop pasa los datos por la entrada y recoge lo que imprimen por la salida.

**1. Map (mapper.py)**

Hadoop divide la entrada en *splits*, normalmente un bloque de 128 MB, y lanza un mapper por split: **54 mappers** para los 54 bloques de los 7 CSV. Cada mapper lee su trozo línea a línea y, para cada línea:

- la separa por |;
- descarta la cabecera y las líneas incompletas (menos de 14 campos);
- descarta las zonas extranjeras (origen que no empieza por dos dígitos);
- convierte la columna viajes (posición 13) a número;
- emite un par **(clave, valor) = (provincia de origen | fecha, viajes)**, separado por un tabulador.

Por ejemplo, la línea `20221026|…|20069|…|3.52|…` (un viaje con origen en Donostia) se convierte en el par `20|20221026  3.52`. Los contadores del job (`evidencias/mr_streaming.log`) muestran 78.340.913 registros leídos (las 78.340.906 filas más las 7 cabeceras) y 77.980.522 pares emitidos, sin las zonas extranjeras.

**2. Shuffle & sort (lo hace Hadoop)**

Hadoop reparte los pares por clave, de modo que todos los pares de una misma provincia y día llegan al mismo reducer (hay 2), y los **ordena por clave**. Esta fase no se programa: la hace el framework. El job no usa combiner, así que los 77,98 millones de pares se escriben a disco y viajan por la red.

**3. Reduce (reducer.py)**

En Hadoop Streaming el reducer no recibe grupos (clave, [valores]), sino líneas sueltas ya ordenadas. Por eso recorre las líneas guardando la clave actual y un acumulador:

- si la clave es la misma que la anterior, suma el valor al total;
- si cambia, emite la clave anterior con su total y empieza a acumular la nueva;
- al terminar, emite la última clave.

La misma agregación se hace en Spark con RDD: map para emitir (provincia | fecha, viajes) y reduceByKey para sumar. reduceByKey combina en cada partición antes del shuffle, que es lo que haría un combiner (el job de Streaming no lo usa). Los dos dan exactamente las mismas **364 claves** (52 provincias × 7 días) con los mismos totales.

Los resultados del trabajo de Map Reduce de Hadoop Frente a los API de RDDs de Spark se pueden apreciar abajo (`metrics.json → c_mapreduce_vs_rdd`, `mr_streaming_time.json`):

| | **Hadoop Streaming** | **Spark RDD** |
|---|---|---|
| Tiempo de ejecución | 312,9 s | 56,9 s (mediana de 3) |
| Líneas de código | 43 (mapper.py + reducer.py) | 8 (una función en src/mr_rdd.py) |
| Tareas | 54 map + 2 reduce | 54 tareas por stage |
| Datos que cruzan el shuffle | 77,98 M pares | ≈ 176 KB (combina en el map) |
| Resultado | 364 claves | 364 claves, idénticas |

En Hadoop este trabajo tarda unas 5,5 veces más que en Spark. Cada map escribe su salida en disco y cada reduce la lee del disco, y ir al disco y volver es lo que más tarda; además, YARN arranca una JVM y un proceso de Python para cada una de las 54 tareas map. Spark mantiene los datos en memoria y reutiliza los mismos executors durante toda la aplicación.

En complejidad, la versión RDD es una función de 8 líneas, frente a las 43 de mapper.py y reducer.py. Spark ofrece operaciones de alto nivel (filter, join, groupBy, reduceByKey) y no ejecuta cada transformación al momento: construye un DAG y solo lo ejecuta cuando se pide una acción, lo que le permite optimizar el plan completo.

Aun así los resultados coinciden: la comparación con un `fullOuterJoin` por clave da 364 claves, ninguna que aparezca solo en una de las dos versiones y una diferencia máxima de 0,0005 viajes, debida al redondeo (`evidencias/resultados/c_mr_streaming_provincia_fecha.tsv` y `c_rdd_provincia_fecha.tsv`).

En la Figura 7 se ve el job terminado en YARN y los stages de la versión RDD en la Spark UI.

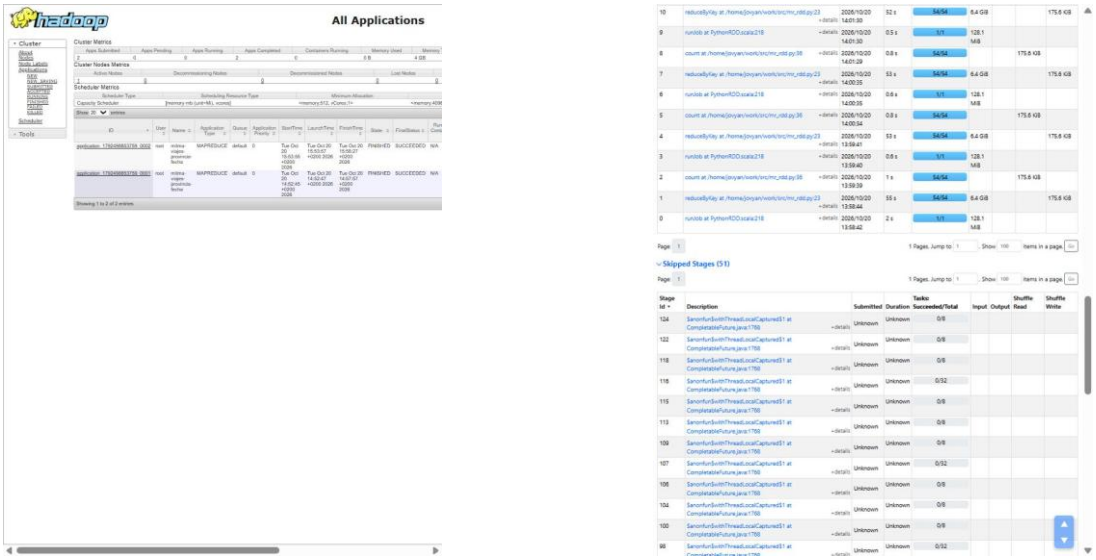



**Figura 7.** *Izquierda: YARN (`localhost:8088`) con el job `mitma-viajes-provincia-fecha` en estado SUCCEEDED. Derecha: Spark UI, stages del RDD: cada repetición lee los 6,87 GB de raw (6,4 GiB en la UI) con 54 tareas y escribe solo 175,6 KiB de shuffle.*

# 4. Procesamiento con Spark

## 4.1 ETL

La ETL transforma los datos de raw en la zona curated en ocho pasos.

1. **Lectura.** Los CSV se leen con un esquema explícito (StructType), con cabecera y separador |. Así los tipos son correctos sin que Spark tenga que adivinarlos, y los códigos de municipio se leen como texto. Resultado: 78.340.906 filas.
2. **Tipado.** La columna fecha pasa de texto a fecha con to_date("yyyyMMdd"), y periodo a número entero, para poder filtrar y particionar por fecha. Antes y después: 78.340.906 filas, ninguna fecha nula.
3. **Nulos.** Se exige que origen, destino, viajes, fecha y periodo no sean nulos, porque un flujo sin origen, sin destino o sin número de viajes no significa nada. Antes y después: 78.340.906 filas.
4. **Rangos.** Se comprueba que periodo esté entre 0 y 23, que viajes sea mayor que 0, que viajes_km sea mayor o igual que 0 y que la fecha esté dentro de la semana estudiada. Así se detectan valores imposibles o de otra semana. Antes y después: 78.340.906 filas, ninguna enviada a cuarentena.
5. **Duplicados.** Se aplica dropDuplicates sobre las 13 dimensiones del flujo, porque un mismo flujo no puede contarse dos veces. Antes y después: 78.340.906 filas, ningún duplicado.
6. **Zonas extranjeras.** Las filas con origen o destino en Francia, Portugal o "externo" no se eliminan: se marcan con las columnas origen_extranjero y destino_extranjero. Son datos válidos, y cada análisis decide si los usa. Afecta a 704.364 filas (0,90 %), que se conservan.
7. **Supresiones.** Los valores edad = NA y sexo = NA se tratan como una categoría más, no como datos ausentes, porque son supresiones de MITMA por privacidad. Afecta a 23.674.348 filas en edad (30,2 %) y 40.612.444 en sexo (51,8 %), que se conservan.
8. **Columnas derivadas.** Se añaden cod_prov_origen y cod_prov_destino, que son los dos primeros dígitos del código INE del municipio. Permiten hacer el join con la tabla de provincias sin modificar la tabla de MITMA.

En total, las 78.340.906 filas de raw cumplen todas las reglas: 0 van a cuarentena, 0 son duplicadas y todos los orígenes españoles tienen nombre en la tabla auxiliar. La zona curated conserva el 100 % de las filas y de los viajes (883,8 millones). Las reglas son la red que detectaría un cambio en la fuente: si las filas inválidas superan el 1 %, el pipeline se detiene. Todas estas cifras están en `metrics.json` (`etl_1_entrada` a `etl_4_tabla_auxiliar`).

## 4.2 Preguntas analíticas

Las tres pregunta analiticas fueron las siguientes y con sus respectivos resultados:

**P1 · Viajes medios diarios por provincia de origen y tipo de día (agrupación)** Tiene dos niveles de agregación: primero el total de viajes por provincia y día, después la media de esos totales según el tipo de día (tabla auxiliar calendario). Usa la API de DataFrames, con dos joins y `avg(when(...))`.

Resultado: 52 filas, en `resultados/d2_p1_provincia_tipo_dia.csv` (se muestran las cinco primeras):

| Provincia | Laborable | Fin de semana | Festivo 1-nov | Finde vs lab. | Festivo vs lab. |
|---|---|---|---|---|---|
| Madrid | 20.426.994 | 15.875.373 | 14.986.665 | -22,3 % | -26,6 % |
| Barcelona | 17.751.202 | 13.990.594 | 12.780.486 | -21,2 % | -28,0 % |
| Valencia/València | 7.429.863 | 6.006.807 | 5.498.337 | -19,2 % | -26,0 % |
| Alicante/Alacant | 5.748.759 | 4.862.469 | 4.509.760 | -15,4 % | -21,6 % |
| Sevilla | 5.410.269 | 4.341.207 | 4.179.734 | -19,8 % | -22,7 % |

En el conjunto de las 52 provincias, un fin de semana tiene un 18,3 % menos de viajes que un laborable y el festivo un 23,2 % menos: el 1 de noviembre se parece más a un domingo que a un sábado. La caída es mayor donde hay mucha movilidad laboral (Araba/Álava −29,2 %, Barcelona −28,0 %) y menor en provincias rurales como Ávila (−4,6 %) o Soria (−14,7 %).

**P2 · Los 3 municipios que más viajes reciben cada día (ventana de ranking, Spark SQL)** Se hace con `SUM(viajes)` por (fecha, destino) y después `ROW_NUMBER() OVER (PARTITION BY fecha ORDER BY viajes_recibidos DESC)`, sobre vistas temporales. Resultado: 21 filas (7 días × 3), en `resultados/d2_p2_top_destinos_dia.csv`:

| Día | 1.º | 2.º | 3.º |
|---|---|---|---|
| Mié 26 | Madrid (10,67 M) | Barcelona (5,53 M) | València (2,53 M) |
| Jue 27 | Madrid (10,67 M) | Barcelona (5,46 M) | València (2,54 M) |
| Vie 28 | Madrid (10,45 M) | Barcelona (5,39 M) | València (2,55 M) |
| Sáb 29 | Madrid (7,95 M) | Barcelona (4,09 M) | València (2,01 M) |
| Dom 30 | Madrid (7,42 M) | Barcelona (3,49 M) | València (1,74 M) |
| Lun 31 | Madrid (8,75 M) | Barcelona (4,48 M) | Sevilla (2,15 M) |
| Mar 1 (festivo) | Madrid (7,45 M) | Barcelona (3,62 M) | València (1,77 M) |

Madrid y Barcelona ocupan los dos primeros puestos los 7 días. València es tercera todos los días salvo el lunes 31, víspera del festivo, en el que la supera Sevilla. Madrid pasa de 10,7 millones de viajes recibidos un miércoles a 7,4 millones el festivo.

**P3 · Desviación de cada municipio respecto a su media semanal (ventana de comparación + lag)** `avg(viajes_dia)` sobre `Window.partitionBy(destino)` da la media semanal de cada municipio sin colapsar filas, y `lag` sobre la misma ventana ordenada por fecha da la variación respecto al día anterior. Solo se miran los municipios que reciben al menos 20.000 viajes al día (`UMBRAL_MUNICIPIO_GRANDE`): 882 municipios × 7 días = 6.174 filas, en `resultados/d2_p3_desviacion_municipio_dia.csv`. Resumen por día (`d2_p3_resumen_por_dia.csv`):

| Día | Mediana de la desviación | Mínimo | Máximo | Municipios a más de un 20 % por debajo |
|---|---|---|---|---|
| Mié 26 | 8,7 % | -40,0 % | 39,9 % | 3 |
| Jue 27 | 8,7 % | -39,2 % | 40,5 % | 2 |
| Vie 28 | 12,5 % | -27,7 % | 37,9 % | 1 |
| Sáb 29 | -4,4 % | -29,0 % | 30,3 % | 7 |
| Dom 30 | -14,0 % | -50,1 % | 45,8 % | 185 |
| Lun 31 | 2,4 % | -19,0 % | 38,6 % | 0 |
| Mar 1 (festivo) | -14,6 % | -34,2 % | 34,8 % | 145 |

El festivo se comporta como un domingo: la mediana cae un 14,6 % y 145 municipios quedan más de un 20 % por debajo de su media. Las mayores caídas del 1 de noviembre son de municipios industriales, como Almussafes (−34,2 %, `d2_p3_mayores_caidas_festivo.csv`). El lunes 31 queda solo un 2,4 % por encima de la media, un efecto puente.

## 4.3 Physical plan

Para la pregunta 1 podemos ver el siguiente physical plan (`evidencias/plan_d2_p1.txt`, Figura 8):




**Figura 8.** *Plan físico de P1 (`explain("formatted")`), leído de abajo arriba.*

En total se puede apreciar como 3 shuffles y 7 stages dado que:

**Paso a paso:**

1. **Tablas pequeñas (4–7 y 10–12).** Se leen el calendario (7 filas) y las provincias (52 filas) y se envían enteras a cada executor con un BroadcastExchange.
2. **Lectura de curated (1–3).** Scan parquet lee solo las 4 columnas necesarias y descarta los orígenes extranjeros en la propia lectura.
3. **Joins (8 y 13).** Dos BroadcastHashJoin cruzan curated con el calendario (por fecha) y con las provincias (por cod_prov_origen) en el mismo executor, sin mover curated.
4. **Agregación parcial (15).** Cada executor suma sus viajes por provincia y día antes de enviar nada, como un combiner.
5. **Shuffle 1, Exchange (16), → total por provincia y día (17–18).** Salen 364 filas.
6. **Shuffle 2, Exchange (19), → media por tipo de día (20–21).** Salen 52 filas, una por provincia.
7. **Shuffle 3, Exchange (22), → Sort (23).** Ordena las provincias de mayor a menor media laborable.

**Shuffles: 3** (Exchange 16, 19 y 22), todos después de los joins y sobre datos ya agregados, de pocos kilobytes. Además hay 2 BroadcastExchange (7 y 12), que solo envían 7 y 52 filas. Los 78 millones de filas de curated no viajan nunca por la red.

**Stages: 7.** Cada Exchange corta el plan: la difusión del calendario y la de las provincias, la lectura de curated con los dos joins y la agregación parcial, el total por provincia y día, la media por tipo de día, el orden final y un stage corto de muestreo que Spark lanza para decidir los rangos del orden.

## 4.4 Optimización

Para comprobar la optimización lo que se hizo fue probar reemplazar broadcast con sort merge en la P1. Dado que Spark eligió broadcast por defecto se decidió comprobar cómo es la forma óptima de hacer el join.

La P1 cruza una tabla enorme con dos tablas diminutas. Con un sort-merge join, Spark tiene que repartir y ordenar las dos tablas por la clave del join, lo que obliga a mover los millones de filas por la red una vez por cada join. Con un broadcast join, la tabla pequeña se copia a cada executor y la grande se queda donde está. Por eso es la optimización adecuada cuando una tabla es mucho más pequeña que la otra. Para medirlo, se desactiva el broadcast con `spark.sql.autoBroadcastJoinThreshold = -1` y después se vuelve al valor por defecto (10 MB), con 3 ejecuciones de cada versión y la mediana (`metrics.json → d3_optimizacion_broadcast`).

El plan con sort merge (`plan_d3_antes_sort_merge_join.txt`) y el plan con broadcast (`plan_d3_despues_broadcast_join.txt`) se comparan en la Figura 9:

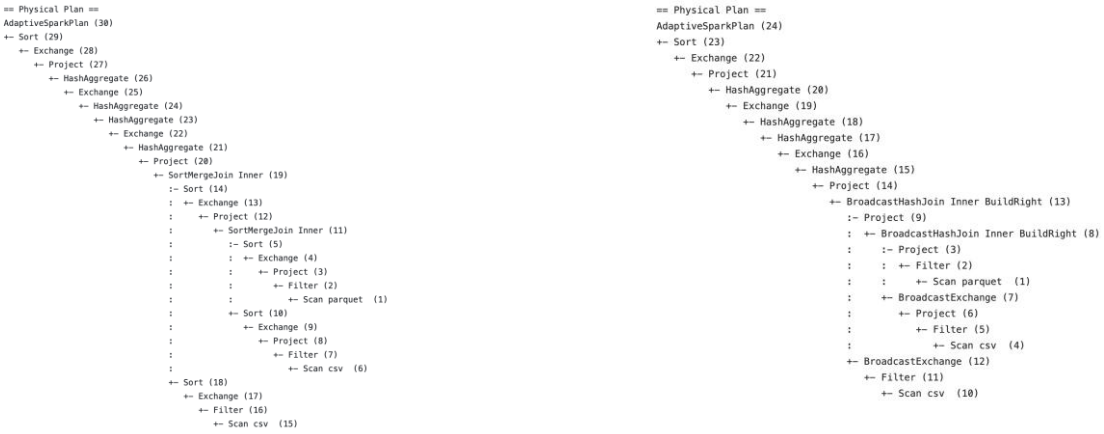



**Figura 9.** *Plan de P1 con sort-merge join (izquierda) y con broadcast join (derecha).*

**Antes (sort-merge join):** los ~78 millones de filas de curated pasan por un Exchange (4) que las reparte por fecha y un Sort (5) para el primer SortMergeJoin (11), y después por otro Exchange (13) y Sort (14) para el segundo, con las provincias. La tabla grande se reparte por la red dos veces, una por cada join: 7 shuffles y 9 stages.

**Después (broadcast join):** los joins pasan a ser BroadcastHashJoin (8) y (13), y las tablas pequeñas se envían enteras a cada executor con un BroadcastExchange (7 y 12). Curated no se mueve y cada partición agrega antes de enviar nada: 3 shuffles y 7 stages.

| | **Sort-merge join (antes)** | **Broadcast join (después)** |
|---|---|---|
| Exchange (shuffles) en el plan | 7 | 3 |
| Stages ejecutados (Spark UI) | 9 | 7 |
| Shuffle write total | 1.209 MB | 0 MB |
| Tiempo (mediana de 3) | 52,4 s | 11,7 s (−77,6 %) |

El broadcast join reduce el tiempo de 52,4 s a 11,7 s porque elimina los shuffles de la tabla grande, que obligaban a escribir y enviar millones de filas por la red solo para encontrarse con unas pocas filas de las tablas auxiliares. En la Spark UI (Figura 10) se ven los stages de las dos versiones: los de sort-merge leen y escriben cientos de MB de shuffle, mientras que con broadcast el shuffle se queda en kilobytes.

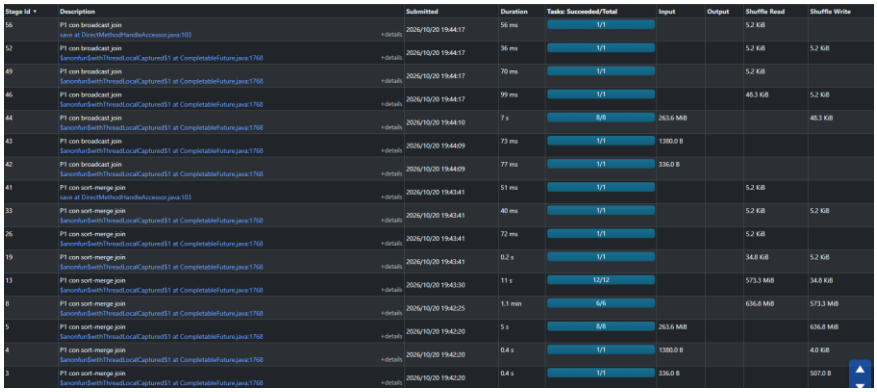



**Figura 10.** *Spark UI, pestaña Stages durante D3: stages de P1 con sort-merge join y con broadcast join, con su shuffle read y write (`evidencias/capturas/d3_sparkui_stages.png`).*

# 5. Integración del pipeline

Con un único comando ejecuta todo el flujo: `.\run_pipeline.ps1` en Windows o `./run_pipeline.sh` en Linux y macOS. Cada paso corre dentro de su contenedor, el script muestra por consola las métricas de cada paso y se detiene en el primer error. La ejecución final completa tardó 46 minutos: 21 en el paso 3 (ETL, formatos y preguntas), 5 en MapReduce y 19 en el paso 5 (RDD, ML y desafío), según `evidencias/tiempos_pasos.csv` y `pipeline_run.log`.

## 5.1 Configuración

Todas las rutas y parámetros están en un solo fichero, `config/pipeline.env`:

- zonas de HDFS;
- fechas de la semana y URL de descarga
- separador, formato de fecha y columna de partición
- umbrales de calidad
- recursos de Spark
- parámetros del modelo

Elegimos el formato .env porque lo leen tanto los scripts bash de los contenedores Hadoop (`source`, sin necesidad de un parser de YAML) como el código Python (`load_config()`, que devuelve un diccionario CONFIG). Ningún fichero de `src/`, `scripts/` o `mapreduce/` contiene rutas de HDFS. El mapper recibe incluso el separador desde la configuración, con `-cmdenv`.

## 5.2 Control de calidad

Cada control tiene una acción definida: registrar en cuarentena, detener, eliminar o avisar. La tabla resume las reglas de `src/etl.py`; en la ejecución final no falló ninguna fila (apartado 4.1), y la consola del paso 3 imprime el número de filas que incumple cada regla.

| Control | Regla | Acción |
|---|---|---|
| Filas inválidas | Fecha fuera de la semana o no parseable, `periodo` fuera de 0–23, origen o destino nulos, `viajes` ⩽ 0 o nulo, `viajes_km` < 0 | **Cuarentena** ( `/quarantine` ) |
| Proporción de inválidas | > 1 % ( `MAX_BAD_RATIO` ) | **Detener** el pipeline ( `assert` , como el "Fail loudly" del Tema 5) |
| Duplicados | Misma clave natural (las 13 dimensiones del flujo) | **Eliminar y registrar** el número |
| Clave de la tabla auxiliar | `id_municipio` repetido | **Registrar** (un join con claves repetidas duplica filas) |
| Municipios sin nombre | > 1 % de orígenes españoles sin match | **Avisar** |
| Nulos y supresiones | Nulos por columna, `edad` / `sexo` = NA | **Registrar** en `metrics.json` |

**Figura 11.** *Controles de calidad del pipeline, con la regla y la acción de cada uno.*

## 5.3 Idempotencia

Al ejecutar el pipeline dos veces deja exactamente el mismo resultado.

**Evidencia:** `evidencias/runs.csv` añade una línea por ejecución de la ETL. Las cinco ejecuciones completas (de 12:42 a 19:34 UTC) dejan exactamente los mismos conteos: 78.340.906 filas en raw, 0 en cuarentena, 78.340.906 en curated y 883.809.802,6 viajes. Las ejecuciones posteriores no volvieron a subir nada a raw (la ingesta detectó los 7 ficheros con el mismo tamaño), y curated sigue teniendo 7 ficheros, no 14.

La estrategia de escritura de cada zona garantiza que una segunda ejecución no duplique ni corrompa nada:

| Zona | Estrategia de escritura | Por qué es idempotente |
|---|---|---|
| raw | `put` solo si el fichero no existe o su tamaño no coincide | Raw es inmutable: nunca se sobrescribe un dato correcto |
| curated | `mode("overwrite")` + `partitionOverwriteMode=dynamic` + `repartition(fecha)` | Se reemplazan solo las fechas presentes; reprocesar un día no borra los demás (Demo 2 del Tema 5) |
| quarantine, results | `mode("overwrite")` | Cada ejecución reemplaza su salida completa |
| salida de MapReduce | `hdfs dfs -rm -r` antes del job | Hadoop exige que la salida no exista |

**Figura 12.** *Estrategia de escritura de cada zona y por qué es idempotente.*

### 5.4 Machine Learning

**Objetivo:** Predecir para cada municipio de origen y cada rango de edad, cuántos viajes salen de media al día utilizando una regresión lineal.

**Feature engineering:** Municipio y edad son categorías, no números. StringIndexer les da un número a cada una y OneHotEncoder las convierte en columnas de 0 y 1 para prevenir *artificial ordering*. Eso da un vector de 2.568 posiciones, casi todo ceros (por eso es "disperso"). El dataset tiene una fila por municipio y rango de edad (12.815 filas) y la variable objetivo es `log1p` de los viajes medios diarios, porque los viajes van de unos pocos a millones.

**Entrenamiento:** `randomSplit([0.8, 0.2], seed=42)` (10.266 filas de train y 2.549 de test) y `VectorAssembler` + `LinearRegression(regParam=0.01)` dentro de un `Pipeline` ajustado **solo con train**, con `handleInvalid="keep"` para que un municipio que solo está en test no rompa el modelo. Se compara con dos líneas base: la media global ($R^2 = 0$ por definición) y la media de cada municipio, sin mirar la edad.

| Modelo (test) | RMSE | MAE | $R^2$ |
|---|---|---|---|
| Baseline 1: media global de train | 1,612 | 1,322 | 0,000 |
| Baseline 2: media del municipio en train | 1,112 | 0,835 | 0,524 |
| **Regresión lineal municipio + edad** | **0,676** | **0,473** | **0,824** |

El modelo (municipio + edad) supera a las dos, y la diferencia con la segunda (0,524 → 0,824) es lo que aporta la edad. **Lectura.** Un MAE de 0,47 en escala log equivale a fallar, de media, por un factor de unas 1,6 veces (e^0,47) en los viajes. Es razonable para un modelo que solo conoce el municipio y la edad. Los coeficientes de edad (`metrics.json → e3_ml.coeficientes_edad`) son coherentes: el grupo de 65 a 100 años viaja bastante menos (−0,87), y el grupo NA es el que más viajes acumula (+1,24), porque agrupa todas las filas en las que MITMA suprime la edad.

## 6. Desafío: el problema de los archivos pequeños

**Objetivo:** medir cuánto cuesta escribir la zona curated en muchos ficheros pequeños y comprobar que la solución aplicada en la ETL, `repartition("fecha")`, lo corrige.

**Por qué aparece en nuestro pipeline:** al escribir con `partitionBy("fecha")`, cada tarea de Spark escribe un fichero en cada carpeta de fecha para la que tiene filas. Después de un shuffle, cada partición contiene filas de los 7 días, así que se generan *número de particiones* × *7* ficheros. Con las 200 particiones de shuffle que Spark usa por defecto (`spark.sql.shuffle.partitions`), una sola semana produciría 1.400 ficheros de menos de 1 MB, muy por debajo de los 128 MB de un bloque de HDFS.

**Experimento** (`src/challenge.py`, en el paso 5 del pipeline). Se escribe la misma tabla curada (78.340.906 filas) en tres variantes, en `/tmp/mitma_smallfiles`:

1. **200 particiones**, el valor por defecto de Spark: `repartition(200)` + `partitionBy("fecha")`.
2. **32 particiones**, las que usa el proyecto (`SPARK_SHUFFLE_PARTITIONS` en `config/pipeline.env`).
3. **repartition("fecha")**, lo que hace la ETL: todas las filas de un día van a la misma tarea.

Para cada variante se mide el número de ficheros, su tamaño medio, el tiempo de escritura y el de leer la tabla sumando los viajes por día (igual que en el apartado 2.3: `noop`, 3 repeticiones y la mediana). Resultados (`metrics.json → f3_archivos_pequenos`):

| Variante | Ficheros | Tamaño medio | Tamaño total | Escritura | Lectura | Tareas al leer |
|---|---|---|---|---|---|---|
| 1 · 200 particiones | 1.400 | 0,9 MB | 1.260 MB | 297,7 s | 4,71 s | 54 |
| 2 · 32 particiones | 224 | 5,2 MB | 1.160 MB | 263,8 s | 1,28 s | 17 |
| 3 · `repartition("fecha")` | 7 | 137,0 MB | 959 MB | 243,6 s | 1,15 s | 8 |

**Análisis paso a paso:**

1. **La lectura es 4 veces más rápida con 7 ficheros que con 1.400** (4,71 s frente a 1,15 s), aunque los datos son exactamente los mismos. Leer un fichero tiene un coste fijo: pedir sus metadatos al NameNode, abrirlo y leer el pie del Parquet, donde están el esquema y las estadísticas. Con 1.400 ficheros de 0,9 MB ese coste fijo pesa más que los propios datos.
2. **El número de tareas lo explica.** Spark agrupa ficheros pequeños en una misma tarea hasta llenar 128 MB (`spark.sql.files.maxPartitionBytes`), pero cuenta cada fichero como si ocupara 4 MB más (`spark.sql.files.openCostInBytes`), precisamente por ese coste de abrirlo. Con 1.400 ficheros: (1.260 MB +

1.400 × 4 MB) / 128 MB ≈ 54 tareas. Con 7 ficheros: (959 MB + 7 × 4 MB) / 128 MB ≈ 8 tareas. Coincide exactamente con lo medido: los datos ocupan 1,3 veces más, pero Spark lanza 7 veces más tareas.

3. **Los ficheros pequeños también ocupan más:** 1.260 MB frente a 959 MB, un 31 % más para los mismos datos. Cada fichero Parquet lleva su propio esquema y sus estadísticas, y la compresión funciona peor con pocos valores, porque hay menos repeticiones que aprovechar.
4. **La escritura mejora menos (de 297,7 s a 243,6 s, un 18 %).** En las tres variantes hay un shuffle completo de los 78 millones de filas (el `repartition`), y ese shuffle domina el tiempo. La variante 3 es la más rápida porque cada tarea escribe un único fichero grande de forma secuencial, en vez de abrir y cerrar 7.

**Corrección aplicada:** la ETL escribe curated con `repartition("fecha")` antes de `partitionBy("fecha")`, así que cada día queda en un solo fichero de entre 124 y 153 MB, del orden de un bloque de HDFS (apartado 2.1). El inconveniente es que cada día lo escribe una sola tarea: si un día tuviera muchos más datos que el resto (*data skew*), esa tarea sería el cuello de botella. Con 7 días de tamaño parecido no ocurre.

**Evidencia:** `evidencias/metrics.json` → `f3_archivos_pequenos`, `evidencias/hdfs_count.txt` (`/tmp/mitma_smallfiles`: 1.634 ficheros entre las tres variantes, contando los `_SUCCESS`) y la consola del paso 5 (`pipeline_run.log`), que imprime la tabla.

# 7. Limitaciones

- **Una sola semana:** no hay estacionalidad y P1 compara un único festivo, así que el modelo no puede generalizar a otras épocas del año.
- **Un solo equipo y un solo worker:** los tiempos sirven para comparar alternativas entre sí, no como rendimiento de un clúster real. En el apartado 3, MapReduce y Spark comparten la misma máquina.
- **Calidad de la fuente:** algunos nombres de la tabla de MITMA llegan con caracteres corruptos (`PolinyÍ` por *Polinyà*). Se dejan tal cual porque la zona raw es inmutable.
- **Modelo aditivo:** no tiene interacción municipio × edad, y el rango NA mezcla edades suprimidas. Es una línea base, no un modelo optimizado.
- **Spark 4:** hubo que desactivar el modo ANSI, para que una fecha mal formada vaya a cuarentena en vez de abortar el job, y fijar `dfs.replication=2` en el cliente HDFS de Spark, que escribía con 3 réplicas en la primera ejecución (`evidencias/hallazgo_replicacion_ejecucion1/`).

# 8. Registro de decisiones de diseño

| Decisión | Alternativas | Motivo |
|---|---|---|
| Data Lake por zonas en HDFS | Data Warehouse | Esquema que puede cambiar, reprocesar el histórico, ML sobre datos detallados y coste |
| Lambda solo con capa batch | Kappa; Lambda completa | Ficheros diarios y un dashboard que tolera horas de retraso |
| Configuración en `.env` | .yaml, .json | La leen igual bash (`source`) y Python, sin dependencias |
| Partición por fecha + `repartition(fecha)` | Sin repartition; partición por provincia | Consultas por día (pruning) y un fichero por día, sin ficheros pequeños |
| Cuarentena y detener por encima del 1 % | Borrar las filas inválidas; solo avisar | No se pierde información, y un cambio brusco de la fuente para el pipeline |
| Zonas extranjeras conservadas y marcadas | Eliminarlas | Son viajes reales; solo se excluyen donde no tienen provincia |
| Optimización D3: broadcast join | Caché; reparticionado | Calendario y provincias son diminutas: elimina dos shuffles de 78 M de filas |
| MapReduce sin combiner | Con combiner | Muestra el coste completo del shuffle frente a lo que `reduceByKey` combina por sí solo |
| ML: municipio de origen + edad, objetivo `log1p` | Destino; variables temporales | No tratarlo como serie temporal; los viajes abarcan seis órdenes de magnitud |
| Desafío 3: archivos pequeños | Escalado; tolerancia a fallos | Conecta con la decisión de particionado y se mide con el mismo método que B.2 |

# 9. Referencias


- Ministerio de Transportes y Movilidad Sostenible (MITMA). *Estudio de movilidad con Big Data*, estudios básicos, matriz diaria de viajes por municipios. https://movilidad-opendata.mitma.es/ (zonificación de municipios: `zonificacion_municipios/nombres_municipios.csv`).
- Apuntes de la asignatura Big Data and Cloud Computing, Temas 1 a 5, y entornos de prácticas `hadoop-course` y `spark-course`.